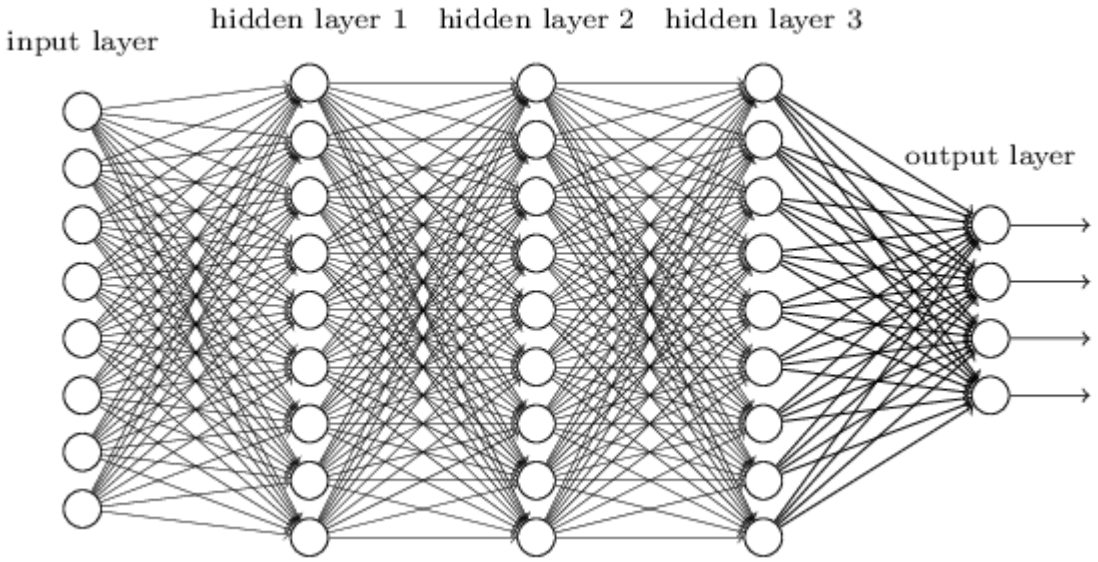
Convolutional Neural Network

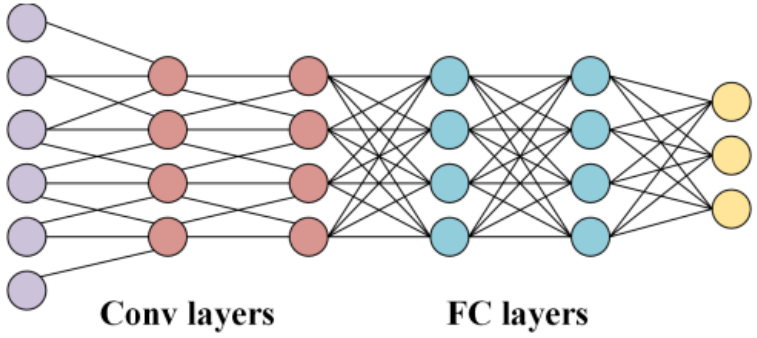
PMI group meeting KWT

LYH Oct 29, 2022

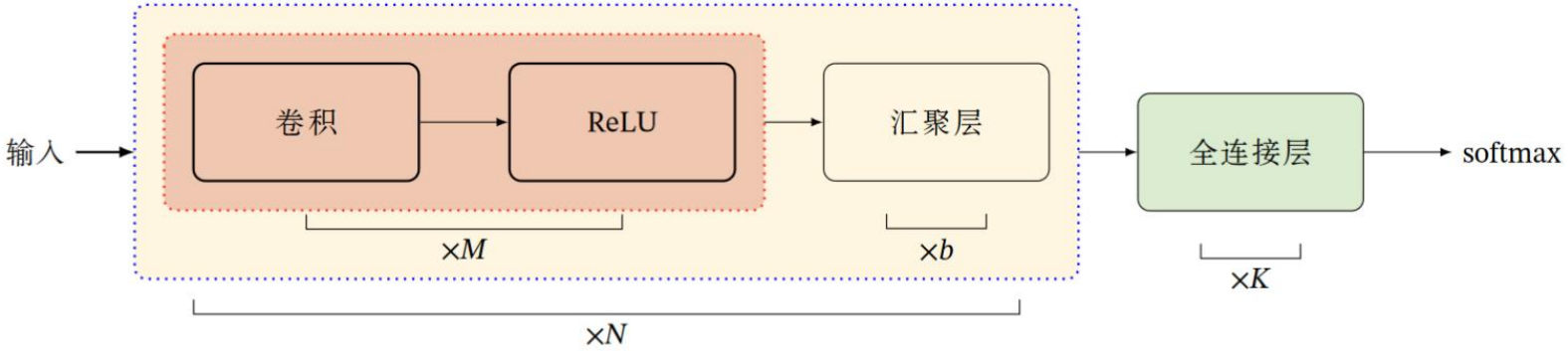
Induction



FNN



CNN



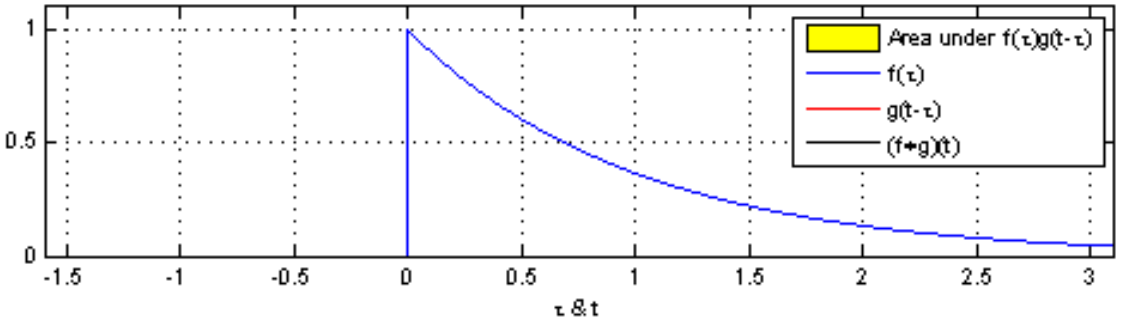
CNN 结构

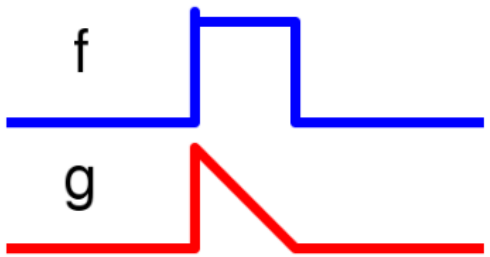
Convolution

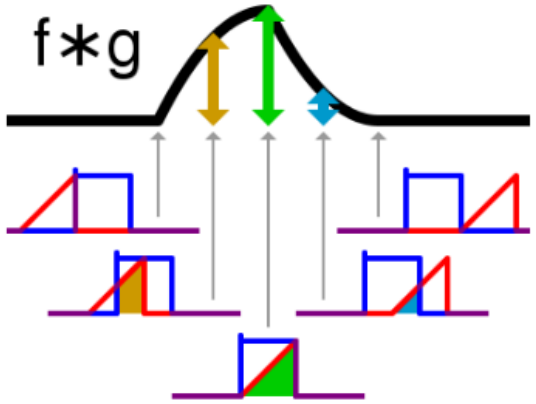
（数学上的）卷积：

$$h(x) = (f * g)(x) \coloneqq \int_{-\infty}^{\infty} f(\tau)g(x-\tau)d\tau$$

$$h(x) = (f * g)(x) \coloneqq \sum_{\tau=-\infty}^{\infty} f(\tau)g(x-\tau)$$







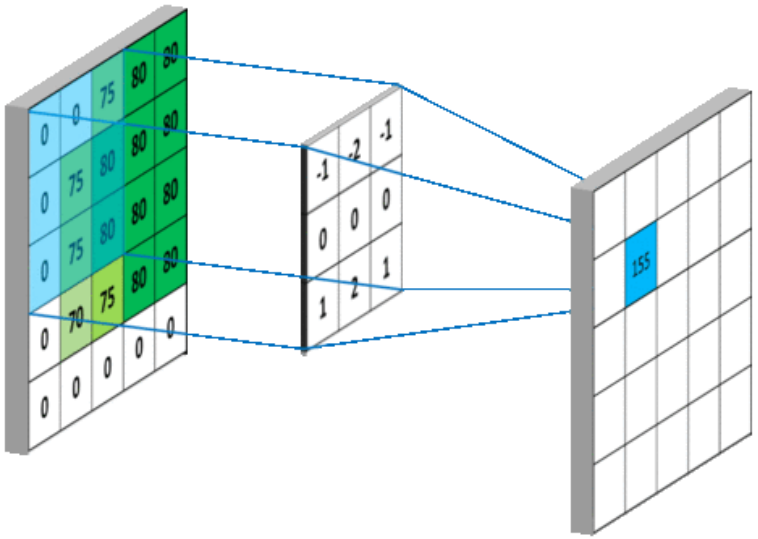
互相关 Cross-correlation

$$(f \star g)(\tau) = \int_{-\infty}^{+\infty} f(t)g(t+\tau)dt$$

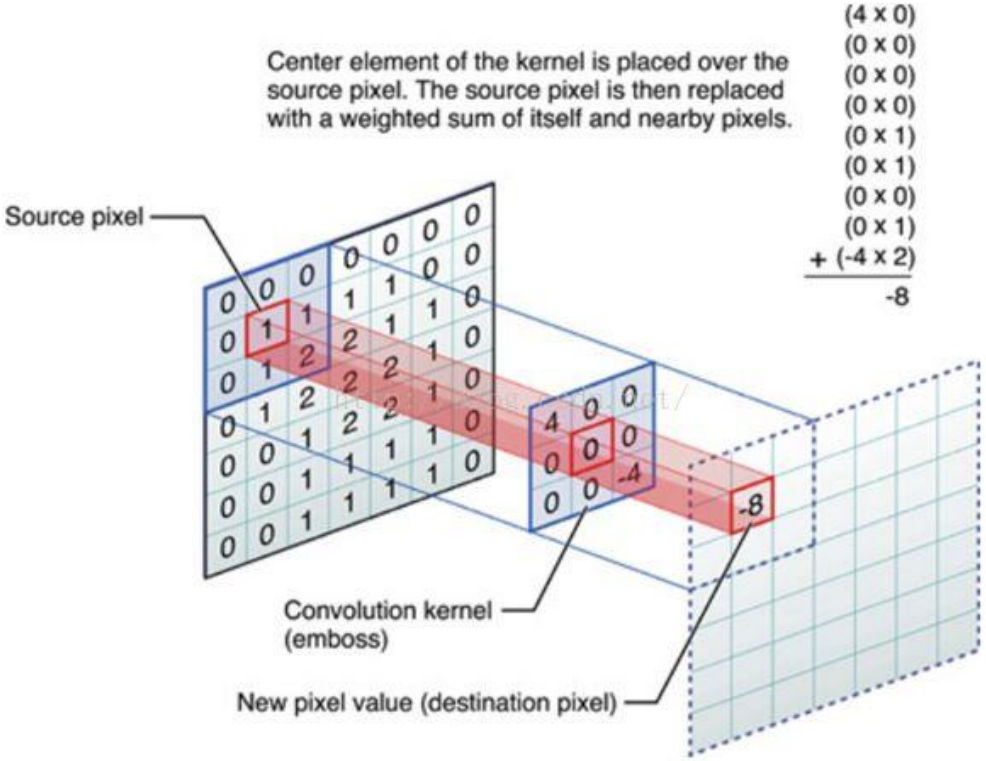
二维卷积（互相关）

$$Y = W * X$$

$$y_{ij} = \sum_{u=1}^{U} \sum_{v=1}^{V} w_{uv} x_{i-u+1,j-v+1}$$



Convolution



$$y_{ij} = \sum_{u=1}^{U} \sum_{v=1}^{V} w_{uv} x_{i+u-1,j+v-1}$$

-1	-1	-1
0	0	0
1	1	1

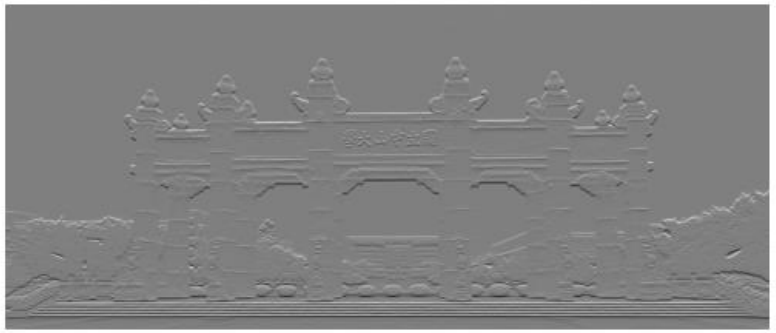
-1	0	1
-1	0	1
-1	0	1

myplot.png

640x480 PNG (24 位颜色) 172.09 kB

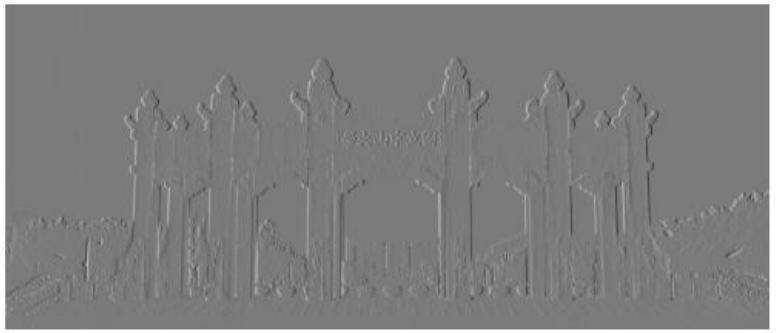
myplot.png

640x480 PNG (24 位颜色) 66.21 kB

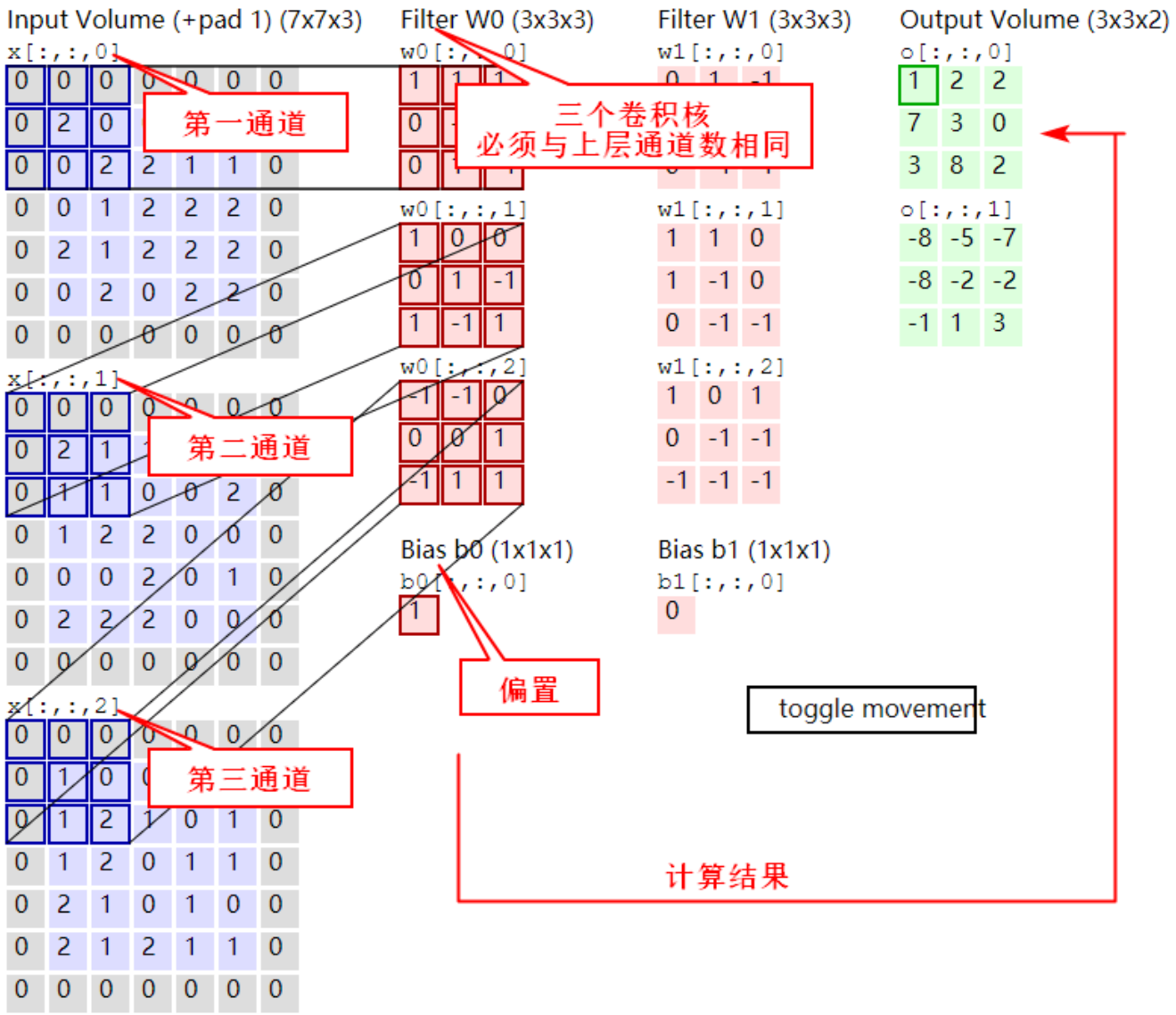


myplot.png

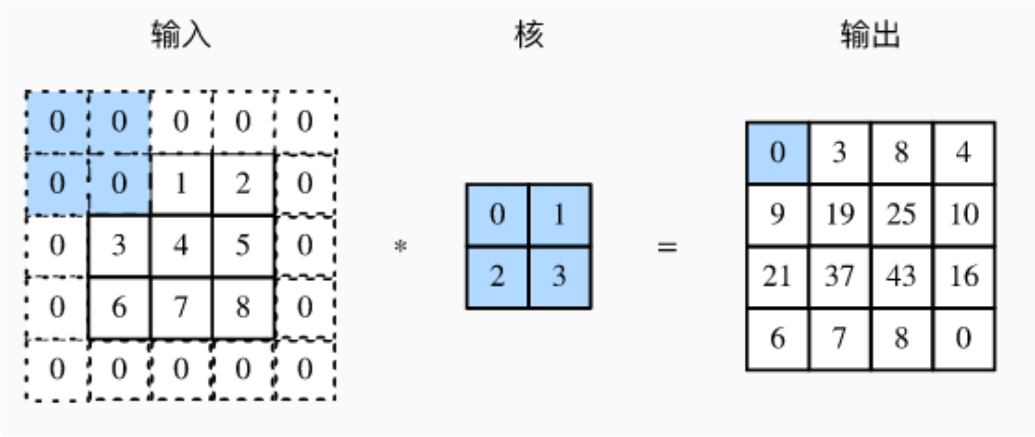
640x480 PNG (24 位颜色) 69.27 kB



3D Convolution



Padding



Stride

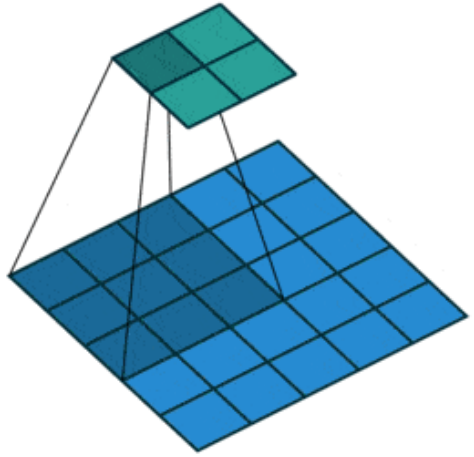
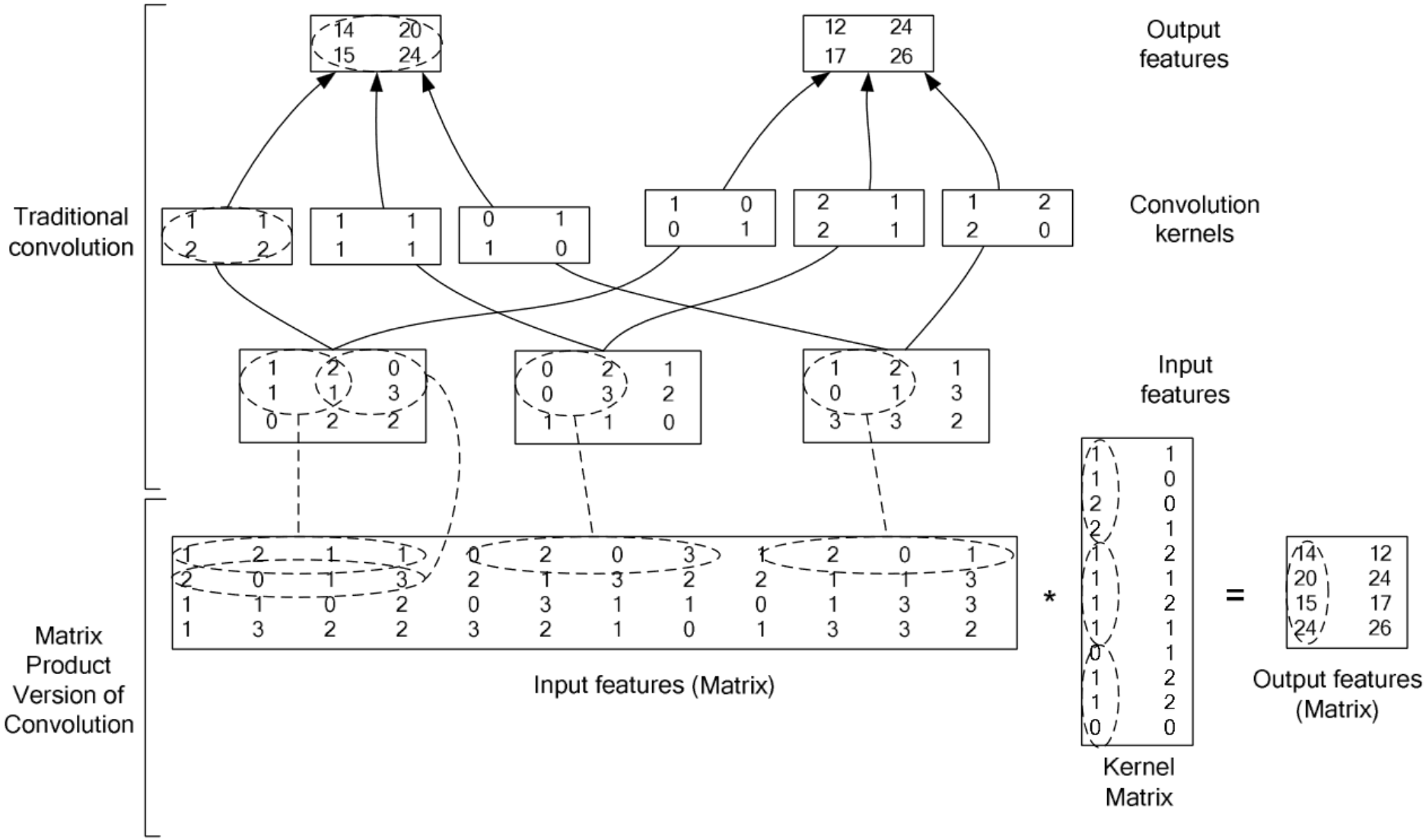


Image To Column (Im2col)

$$y_{ij} = \sum_{u=1}^{U} \sum_{v=1}^{V} w_{uv} x_{i-u+1,j-v+1}$$

```python
# 卷积运算及卷积层的实现
def convolution_2d(W: Matrix, X: Matrix, stride=1):
    Y_list = []

    for k in range(X.row - W.row + 1):
        Y_list.append([])
        for l in range(X.column - W.column + 1):
            y = 0
            for m in range(W.row):
                for n in range(X.row):
                    y = y + W[m][n] * X[stride * k + m][stride * l + n]  # 注意index从0开始，不需要-1
            Y_list[k].append(y)

    return Matrix(Y_list)


def convolution_3d(W: Tensor4, X: Tensor3, stride=1):
    Y_list = []
    for k in range(W.number):
        y_list = []
        for d in range(X.depth):
            y_list.append(convolution_2d(Matrix(W[k][d]), Matrix(X[d]), stride=stride).list)
        interim_result = Tensor3(y_list)
        Y_list.append(interim_result.combine().list)
    return Tensor3(Y_list)
```

Image To Column (Im2col)

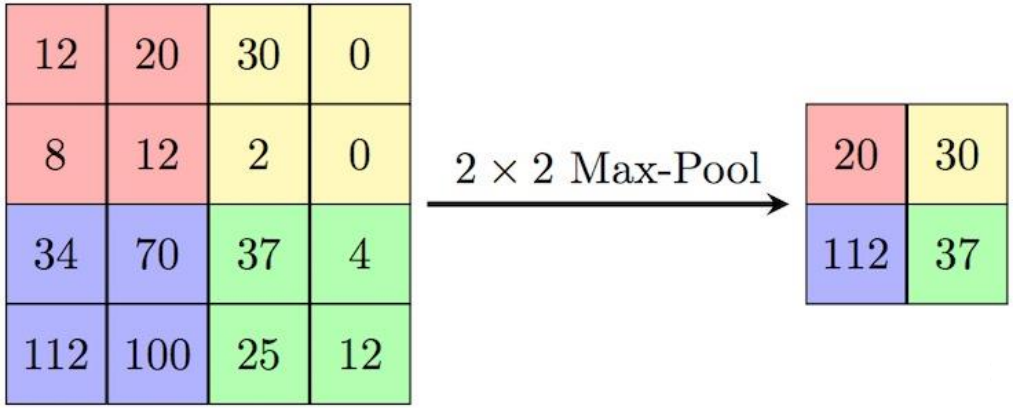


Chellapilla, K., Puri, S., & Simard, P.Y. (2006). High Performance Convolutional Neural Networks for Document Processing.

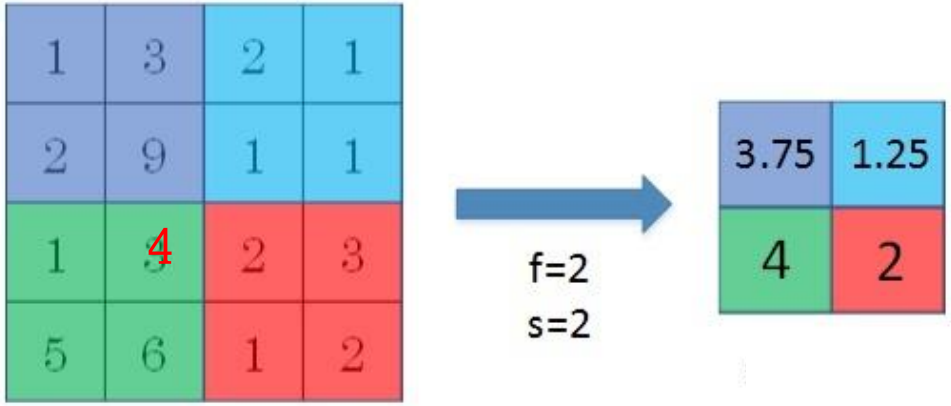
Image To Column (Im2col)

```python
def forward(self, X: list):
    """ 卷积层的前向传播

    :param X: 输入应该是一个四维张量（图片数，通道数，行数，列数）
    :return: N张图片进行一次卷积后的结果，是一个四维张量（图片数，卷积核数，行数，列数）
    """
    X_ndarray = np.array(X)
    N, C, H, W = X_ndarray.shape
    out_h = (H - self.W_shape[3]) // self.stride + 1
    out_w = (W - self.W_shape[2]) // self.stride + 1
    strides = (*X_ndarray[0][0].strides[:-2], X_ndarray[0][0].strides[-2] * self.stride,
               X_ndarray[0][0].strides[-1] * self.stride, *X_ndarray[0][0].strides[-2:])
    result_list = []
    for i in range(N):
        tmp = []
        for j in range(C):
            A = np.lib.stride_tricks.as_strided(X_ndarray[i][j],
                                                shape=(out_h, out_w, self.W_shape[3], self.W_shape[2]),
                                                strides=strides)
            tmp.append(A.ravel().reshape(-1, self.W_shape[3] * self.W_shape[4]))
        result = (np.dot(np.concatenate(tuple(tmp), axis=1), self.W_mat) + self.B).T.reshape(-1, out_w, out_h)
        result[result < 0] = 0  # ReLU 函数
        result_list.append(result.tolist)

    return result_list
```

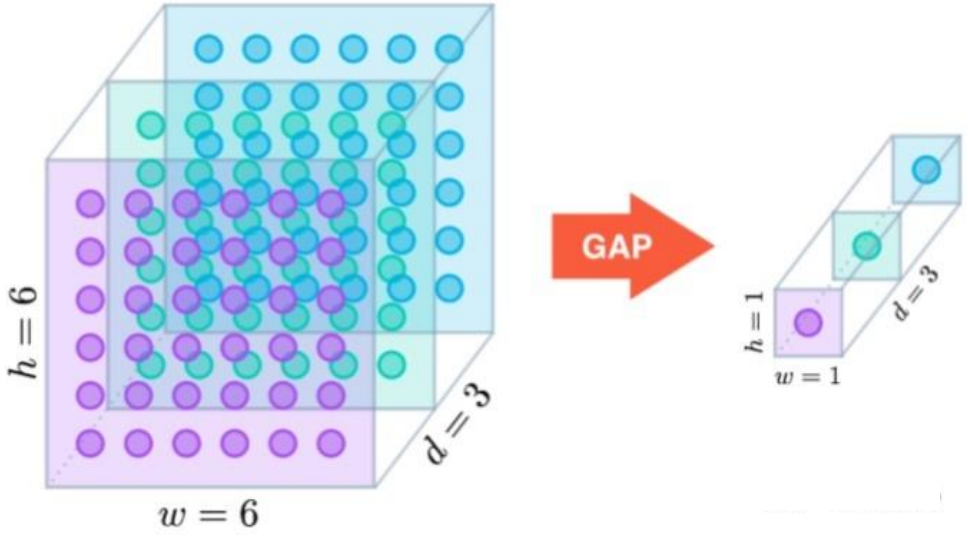
Pooling



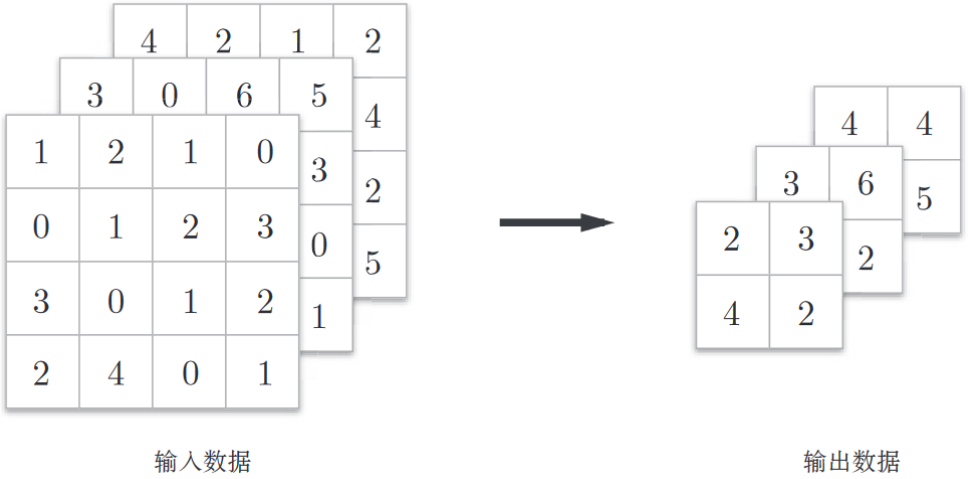
Max Pooling



Average Pooling



Global Average Pooling



3D Pooling

Forward propagation

input $X^1_{I\times J\times K}$

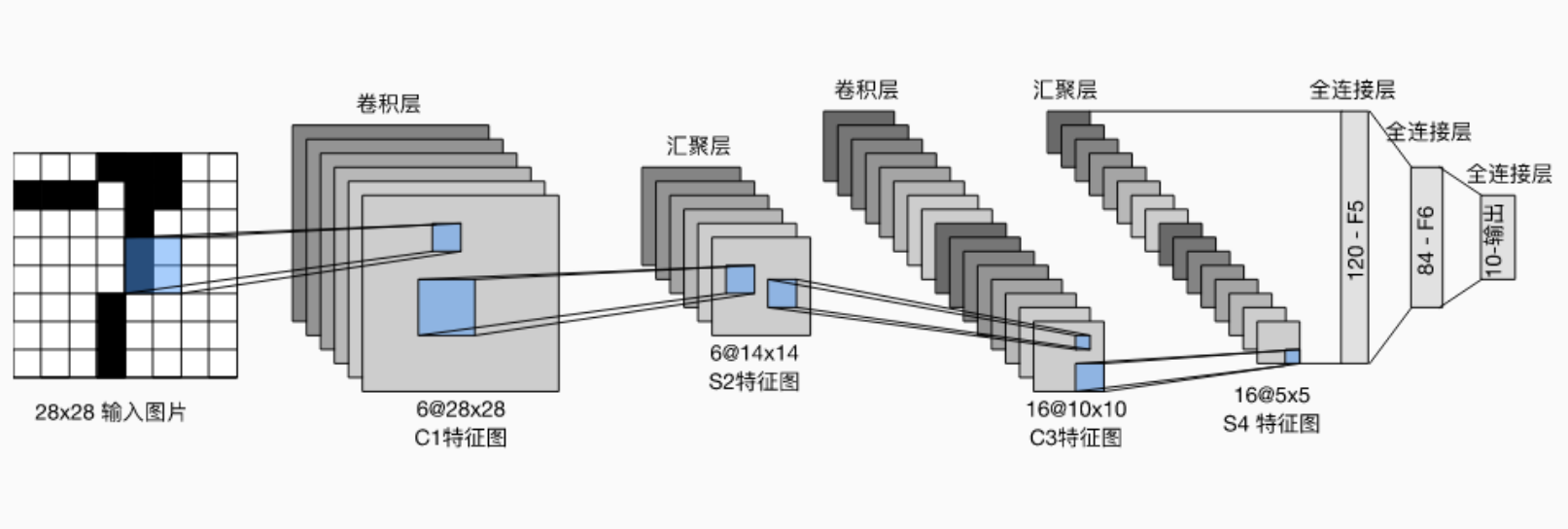
Convolution Layer

$$Z^L = W^L * X^{L-1} + b^L$$
$$X^L = \sigma(Z^L) = ReLU(Z^L)$$

X^L

Pooling Layer

$$X^{L+1} = pooling(X^L)$$



LeNet-5（LeCun等，1998）

$$\mathrm{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

重复N次

Fully Connected Layer

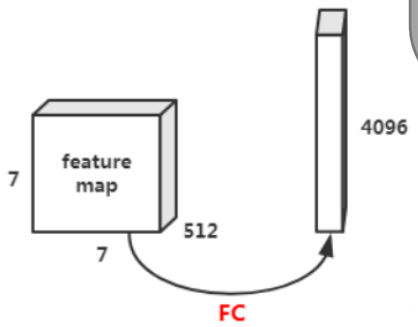
$$Z^l = W^l \cdot X^{l-1} + b^l$$
$$X^l = \sigma(Z^l)$$

$$Z^{l+1} = W^{l+1} \cdot X^l + b^{l+1}$$

Z^{l+1}

Output = $SoftMax(Z^{l+1})$

$$\mathrm{Softmax}(x) = \frac{e^{x_i}}{\sum_i e^{x_i}}$$

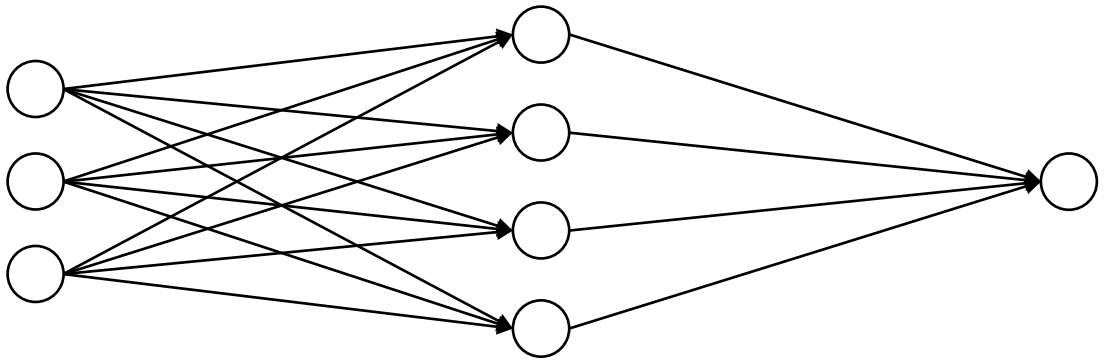


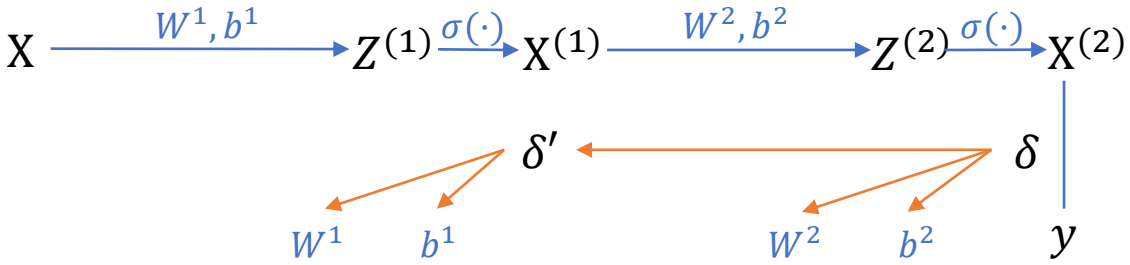
Backpropagation

Fully Connected Layer

$$Z^l = W^l \cdot X^{l-1} + b^l$$
$$X^l = \sigma(Z^l)$$

$$Z^{l+1} = W^{l+1} \cdot X^l + b^{l+1}$$
$$X^{l+1} = \sigma(Z^{l+1})$$





Cost function $C = f(X^L) = f(\sigma(Z^L))$

Error $\delta^L \equiv \dfrac{\partial C}{\partial Z^L} = \dfrac{\partial C}{\partial X^L}\dfrac{\partial X^L}{\partial Z^L} = f'(X^L)\,\sigma'(Z^L)$

$$\delta^{L-1} \equiv \frac{\partial C}{\partial Z^{L-1}} = \frac{\partial C}{\partial Z^L}\frac{\partial Z^L}{\partial X^{L-1}}\frac{\partial X^{L-1}}{\partial Z^{L-1}}$$
$$= \delta^L \cdot h'(X^{L-1}) \odot \sigma'(Z^{L-1})$$
$$= W^{L^T}\delta \odot \sigma'(Z^{L-1})$$

$$\frac{\partial C}{\partial W^L} = \frac{\partial Z^L}{\partial W^L}\frac{\partial C}{\partial Z^L} = X^{L-1}\delta^L$$

$$\frac{\partial C}{\partial b^L} = \frac{\partial Z^L}{\partial b^L}\frac{\partial C}{\partial Z^L} = \delta^L$$

$$W^L \leftarrow W^L - \eta\frac{\partial C}{\partial W^L}$$

$$b^L \leftarrow b^L - \eta\frac{\partial C}{\partial b^L}$$

Backpropagation

Convolution Layer

$$Z^L = W^L * X^{L-1} + b^L$$
$$X^L = \sigma(Z^L) = ReLU(Z^L)$$

$$\delta^L \equiv \frac{\partial C}{\partial Z^L} = \frac{\partial C}{\partial X^L}\frac{\partial X^L}{\partial Z^L} = f'(X^L)\,\sigma'(Z^L)$$

$$\delta^{L-1} \equiv \frac{\partial C}{\partial Z^{L-1}} = \frac{\partial X^{L-1}}{\partial Z^{L-1}}\frac{\partial Z^L}{\partial X^{L-1}}\frac{\partial C}{\partial Z^L}$$

$$\frac{\partial C}{\partial W^L} = \frac{\partial C}{\partial Z^L}\frac{\partial Z^L}{\partial W^L}$$

$$\frac{\partial C}{\partial b^L} = \frac{\partial Z^L}{\partial b^L}\frac{\partial C}{\partial Z^L}$$

$$\frac{\partial z_{kl}^L}{\partial x_{ij}^{L-1}} = \frac{\partial \sum_{m=1}^{M}\sum_{n=1}^{N} w_{m,n} x_{k+m-1,l+n-1}}{\partial \sum_{i=1}^{I}\sum_{j=1}^{J} x_{ij}}$$

$$= \frac{\partial \sum_{i=1}^{I}\sum_{j=1}^{J} w_{i-k+1,j-l+1}\, x_{i,j}}{\partial \sum_{i=1}^{I}\sum_{j=1}^{J} x_{i,j}}$$

$$= w_{i-k+1,j-l+1}$$

$$i = k + m - 1 \Rightarrow m = i - k + 1$$
$$j = l + n - 1 \Rightarrow n = j - l + 1$$

$$\frac{\partial z_{kl}^L}{\partial X^{L-1}} = \sum_{i=1}^{I}\sum_{j=1}^{J}\frac{\partial z_{kl}^L}{\partial x_{ij}^{L-1}} = \sum_{i=1}^{I}\sum_{j=1}^{J} w_{i-k+1,j-l+1}^L$$

$$\frac{\partial Z^L}{\partial X^{L-1}} = rot180(W^L) *$$

$$\delta^{L-1} = \frac{\partial X^{L-1}}{\partial Z^{L-1}}\frac{\partial Z^L}{\partial X^{L-1}}\frac{\partial C}{\partial Z^L} = \sigma(Z^{L-1}) \odot (rot180(W^L) * \delta^L)$$

$$\frac{\partial Z^L}{\partial W^{L-1}} = * rot180(X^{L-1})$$

$$\frac{\partial C}{\partial W^L} = \frac{\partial C}{\partial Z^L}\frac{\partial Z^L}{\partial W^L} = \delta^L * rot180(X^{L-1})$$

$$\frac{\partial C}{\partial b^L} = \frac{\partial Z^L}{\partial b^L}\frac{\partial C}{\partial Z^L} = \delta^L$$

$$W^L \leftarrow W^L - \eta\frac{\partial C}{\partial W^L}$$

$$b^L \leftarrow b^L - \eta\frac{\partial C}{\partial b^L}$$

Backpropagation

Pooling Layer

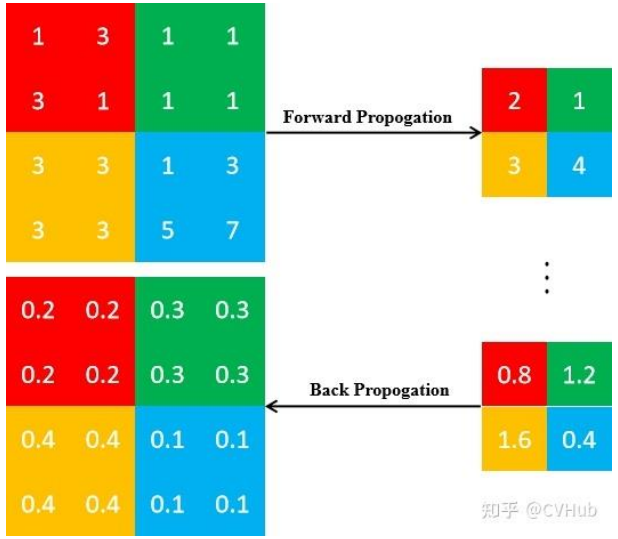
$$X^{L+1} = pooling(X^L)$$

$$Z^L = X^{L-1}$$

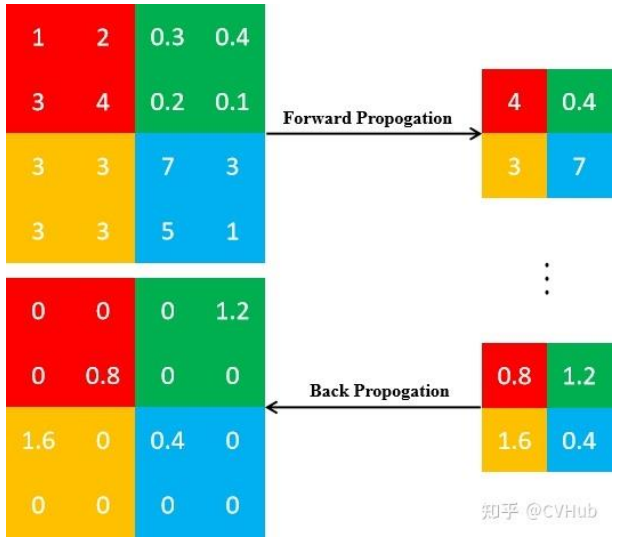
$$X^L = pooling(Z^L)$$

$$\delta^L \equiv \frac{\partial C}{\partial Z^L} = \frac{\partial C}{\partial X^L}\frac{\partial X^L}{\partial Z^L} = f'(X^L)\,\sigma'(Z^L)$$

$$\delta^{L-1} \equiv \frac{\partial C}{\partial Z^{L-1}} = \frac{\partial C}{\partial Z^L}\frac{\partial Z^L}{\partial X^{L-1}}\frac{\partial X^{L-1}}{\partial Z^{L-1}} = \frac{\partial C}{\partial Z^L}\frac{\partial X^{L-1}}{\partial Z^{L-1}} = up_sample(\delta^L)$$



Average Pooling



Max Pooling

Algorithm

input $X^0_{I\times J\times K}$

$Z^1 = W^1 * X^0 + b^1$
$X^1 = ReLU(Z^1)$

$Z^2 = pooling(X^1)$
$X^2 = ReLU(Z^2)$

$Z^3 = W^3 * X^2 + b^3$
$X^3 = ReLU(Z^3)$

$Z^4 = pooling(X^3)$
$X^4 = ReLU(Z^4)$

$Z^5 = W^5 \cdot X^4 + b^5$
$X^5 = \sigma(Z^5)$

$Z^6 = W^6 \cdot X^5 + b^6$
$X^6 = \sigma(Z^6)$

$Z^7 = W^7 \cdot X^6 + b^7$

Output=$SoftMax(Z^7)$

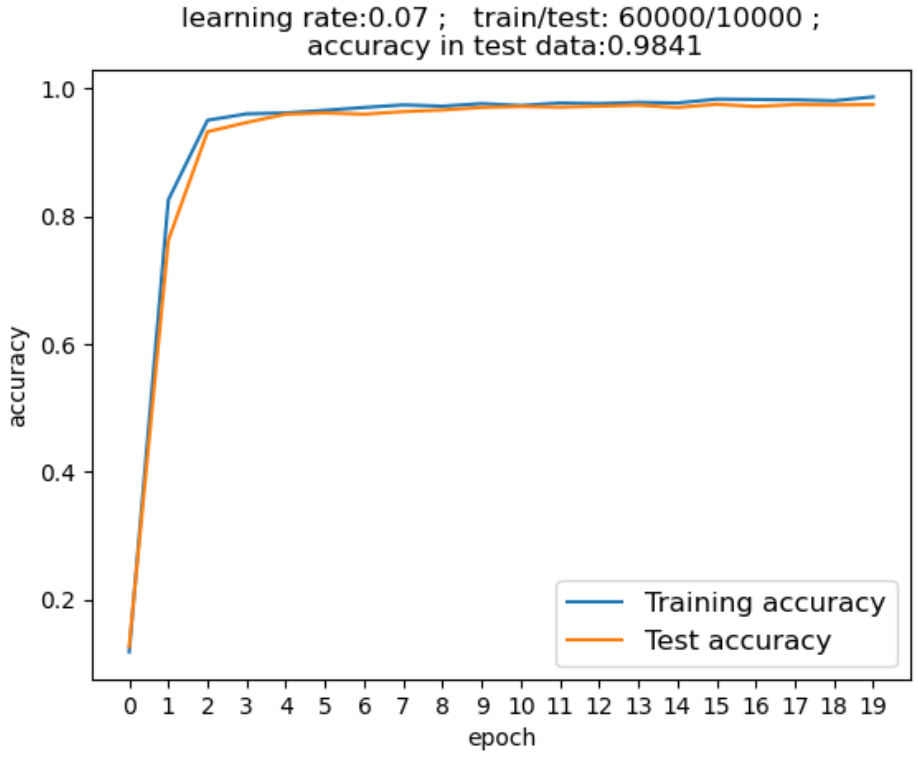
$\delta^7 = label - output$
$\delta^6 = W^{7^T}\delta^7 \odot ReLU'(Z^6)$
$\delta^5 = W^{6^T}\delta^6 \odot ReLU'(Z^5)$
$\delta^4 = W^{5^T}\delta^5 \odot ReLU'(Z^4)$
$\delta^3 = up_sample(\delta^4)$
$\delta^2 = \sigma(Z^2) \odot (rot180(W^3) * \delta^3)$
$\delta^1 = up_sample(\delta^2)$

$W^7 \leftarrow W^7 - \eta X^6\delta^7$
$b^7 \leftarrow b^7 - \eta\delta^7$

$W^6 \leftarrow W^6 - \eta X^5\delta^6$
$b^6 \leftarrow b^6 - \eta\delta^6$

$W^4 \leftarrow W^4 - \eta X^4\delta^4$
$b^4 \leftarrow b^4 - \eta\delta^4$

$W^3 \leftarrow W^3 - \eta\delta^3 * rot180(X^2)$
$b^3 \leftarrow b^3 - \eta\delta^3$

$W^1 \leftarrow W^1 - \eta\delta^1 * rot180(X^0)$
$b^1 \leftarrow b^1 - \eta\delta^1$

```
self.layers['Conv1'] = Convolution(self.params['W1'], self.params['b1'], conv1_param['stride'],conv1_param['pad']
self.layers['Relu1'] = Relu()
self.layers['Pool1'] = Pooling(pool_h=2, pool_w=2, stride=2)

self.layers['Conv2'] = Convolution(self.params['W2'], self.params['b2'], conv1_param['stride'],conv1_param['pad']
self.layers['Relu2'] = Relu()
self.layers['Pool2'] = Pooling(pool_h=2, pool_w=2, stride=2)

self.layers['Affine1'] = Affine(self.params['W3'], self.params['b3'])
self.layers['Relu3'] = Relu()

self.layers['Affine2'] = Affine(self.params['W4'], self.params['b4'])
self.last_layer = SoftmaxWithLoss()
```